

Introduction to CUDA Python using Vibe Coding



Jongsoo Kim

Korea Astronomy and Space Science Institute

The 17th KIAS CAC Summer School:
AI and Parallel Computing

What is CUDA?

Compute Unified Device Architecture: Parallel Computing Platform & Programming Model

Platform

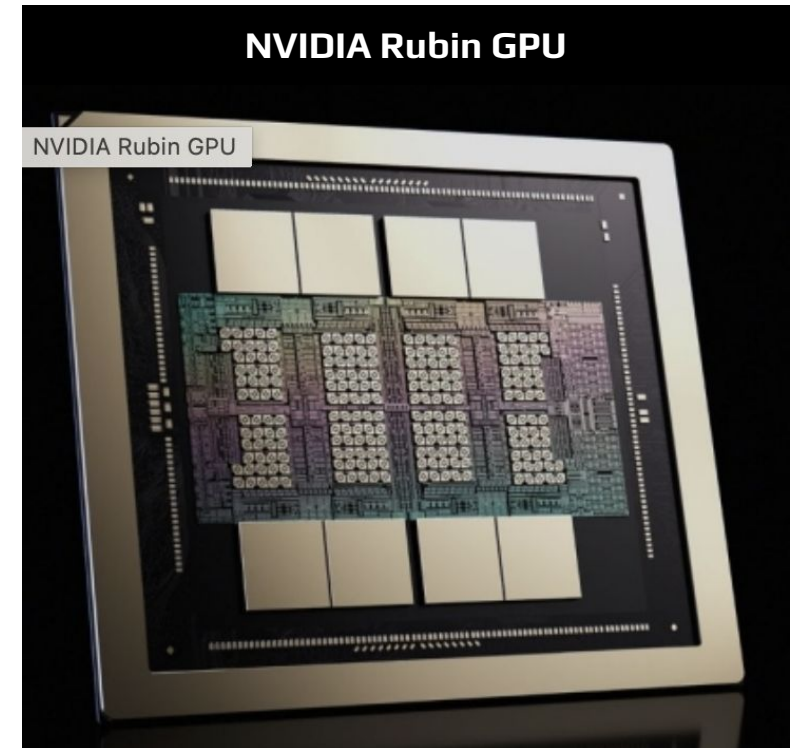
CUDA is a computing platform developed by NVIDIA for general-purpose computing on GPUs.

It enables dramatically increased computing performance by harnessing the power of the graphics processing unit.

Programming

Extends standard languages like C, C++, and Python to support parallel execution.

Developers use "Kernels" to define functions that run thousands of threads simultaneously across GPU cores.



A Brief History of CUDA

- **2006:** NVIDIA introduces CUDA, moving beyond fixed-function pipelines.
- **2007:** Release of CUDA 1.0, enabling C-based GPU programming.
- **2012:** **Kepler** architecture launched; CUDA used for AlexNet (deep learning milestone).
- **2013:** CUDA 5.5 adds ARM support, expanding beyond x86.
- **2014:** **Maxwell** architecture; CUDA 6 introduces Managed Memory.
- **2015:** CUDA 7.5 adds support for 16-bit floating point (FP16).
- **2016:** **Pascal** architecture with **NVLink**; CUDA 8.0 released.
- **2017:** **Volta** architecture introduces **Tensor Cores** for AI acceleration.
- **2019:** CUDA 10.1 enhances cuBLAS and cuFFT performance.
- **2020:** **Ampere** architecture; CUDA 11 introduces Multi-Instance GPU (MIG).
- **2024:** **Blackwell** architecture announced with massive performance leaps.
- **2026:** **CUDA (13.3) Python "First-Class" support integrates across the stack.**
- **2026:** **Rubin** architecture; CUDA celebrates its **20th anniversary** with next-gen breakthroughs.

GB200:
single precision 180 Tflops

H100 SXM:
single precision 60 Tflops

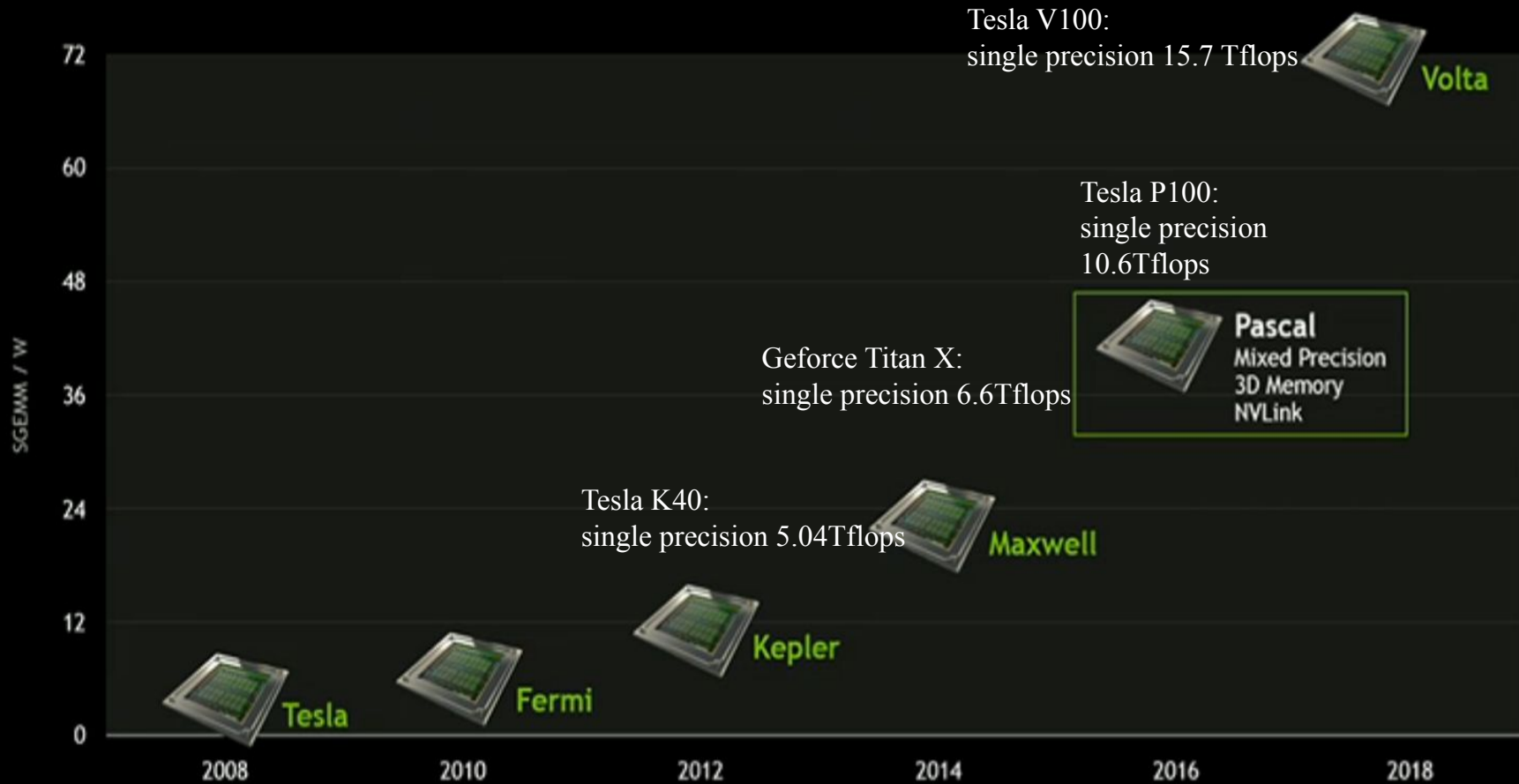
Tesla A100:
single precision 19.5
Tflops

Tesla V100:
single precision 15.7 Tflops

Tesla P100:
single precision
10.6Tflops

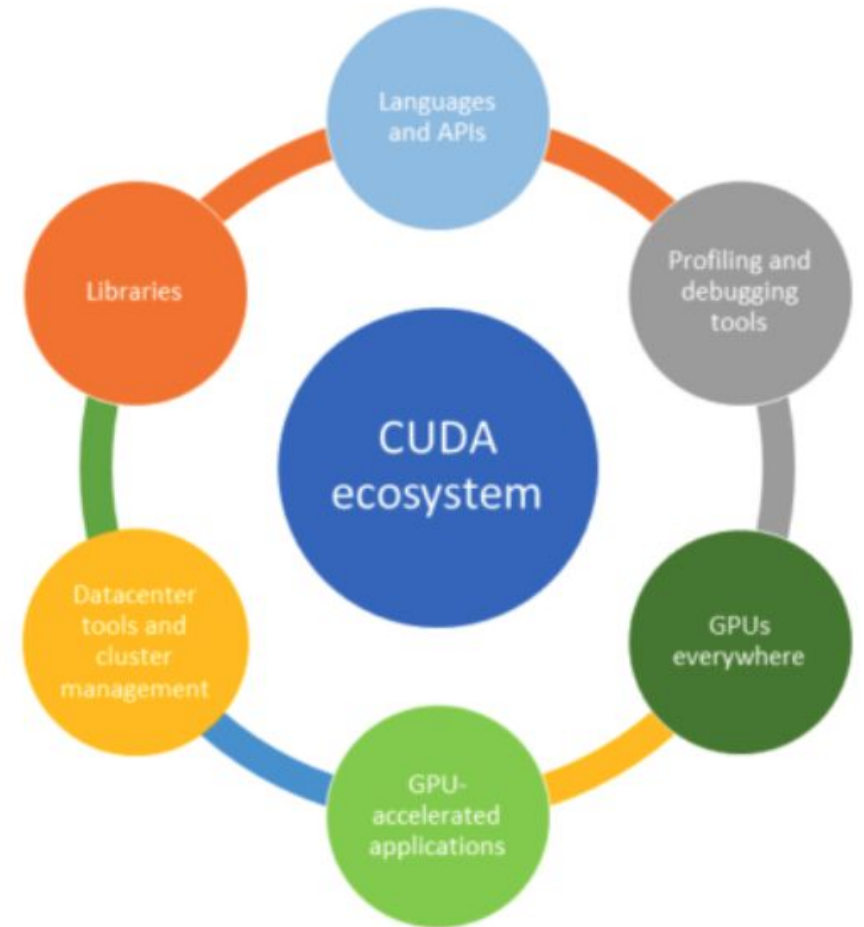
Geforce Titan X:
single precision 6.6Tflops

Tesla K40:
single precision 5.04Tflops

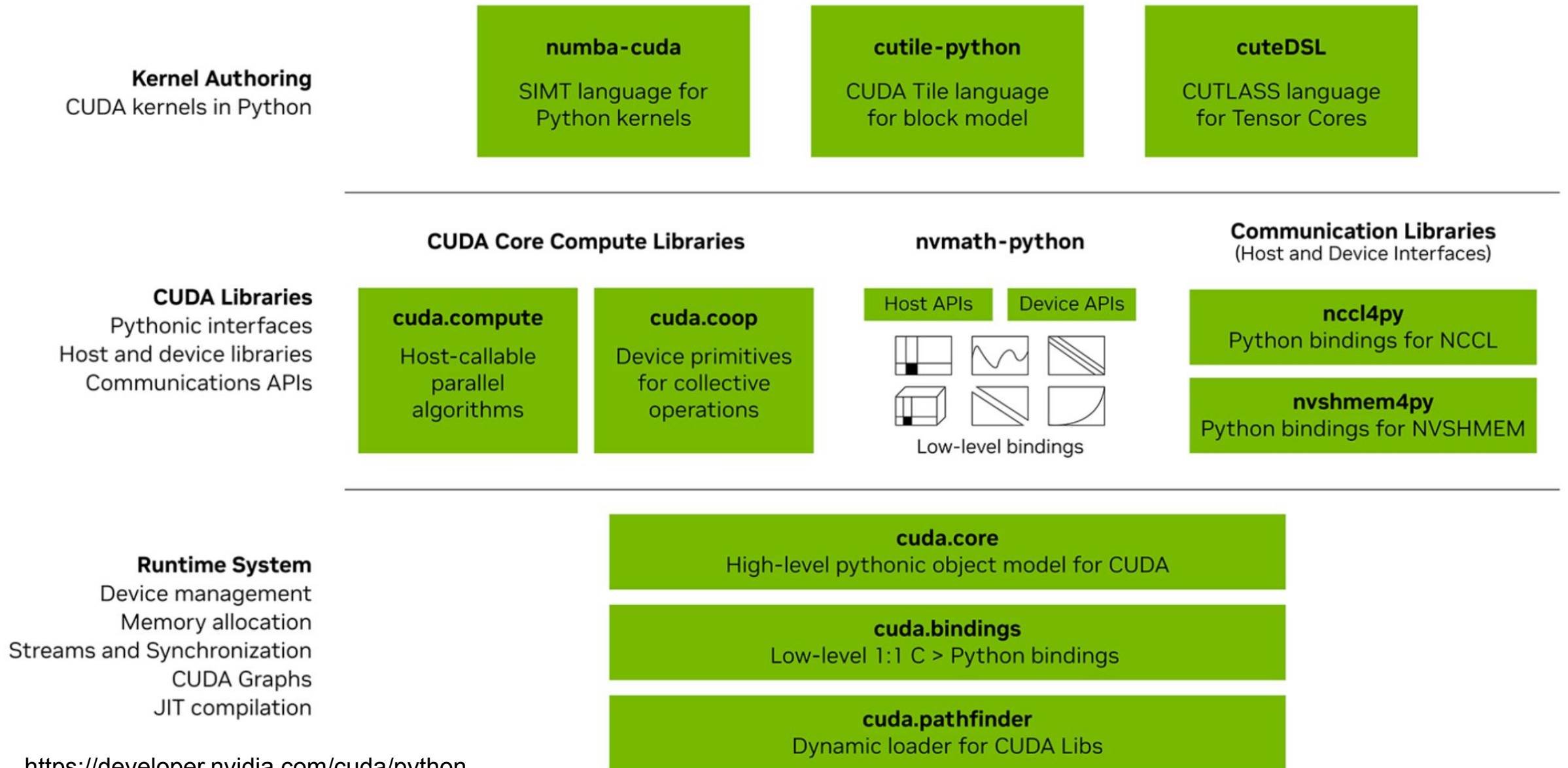


CUDA Ecosystem

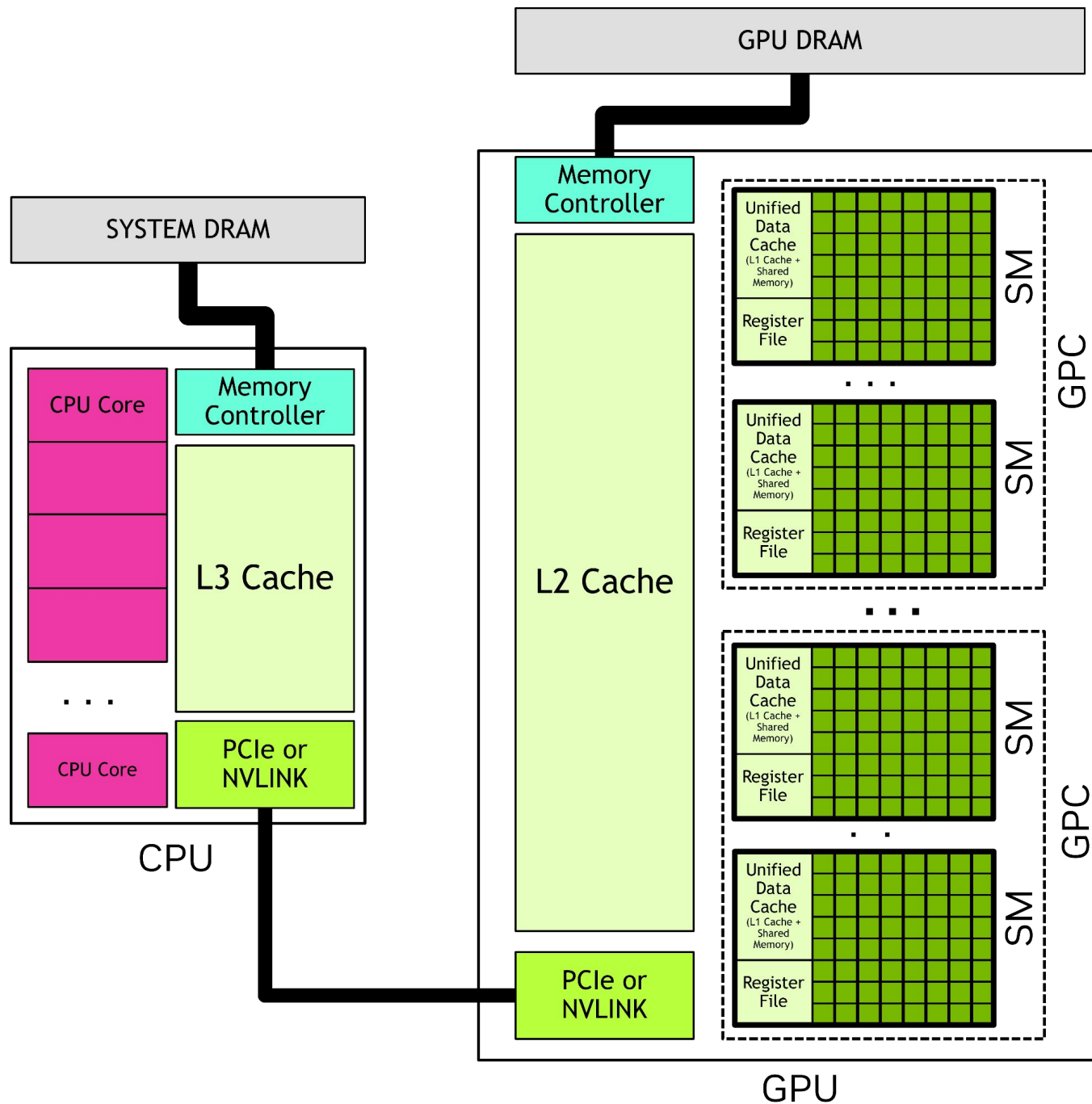
- Programming languages and APIs
 - **CUDA**: C, C++, Fortran, Python, etc.
- Libraries
 - Mathematical libraries: **cuBLAS**, **cuRAND**, **cuFFT**, cuSPARSE, cuTENSOR, cuSOLVER
 - Parallel algorithm libraries: nvGRAPH, Thrust
 - Image and video libraries: nvJPEG, NPP, Optical Flow SDK
 - Communication libraries: **NVSHMEM**, **NCCL**
 - Deep learning libraries: **cuDNN**, **TensorRT**, Jarvis, DALI
 - Partner libraries: OpenCV, FFmpeg, ArrayFire, MAGMA
- Profiling and debugging tools
 - **NVIDIA Nsight**: profiling, tracing, and debugging tool.
 - **CUDA GDB**: extension of Linux GDB, a console-based debugging interface
 - **CUDA-Memcheck**: memory access issues by examining the thousands of threads running concurrently.



Cuda Python Resource (Cuda Toolkit 13.3)



CPU & GPU Architecture



System Architecture

GPU Structure

- SMs: Streaming Multiprocessors contain many functional units.
- GPCs: Graphics Processing Clusters are collections of SMs.
- GPU is a set of GPCs connected to GPU memory.

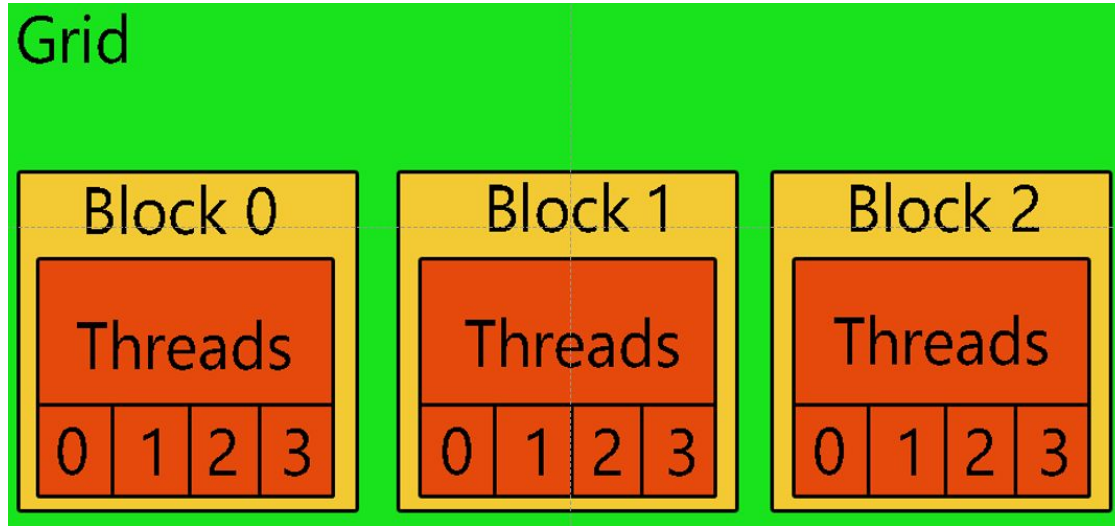
CPU Structure

- Cores: Multiple processing cores with a memory controller.
- System Memory: Connected via the memory controller.

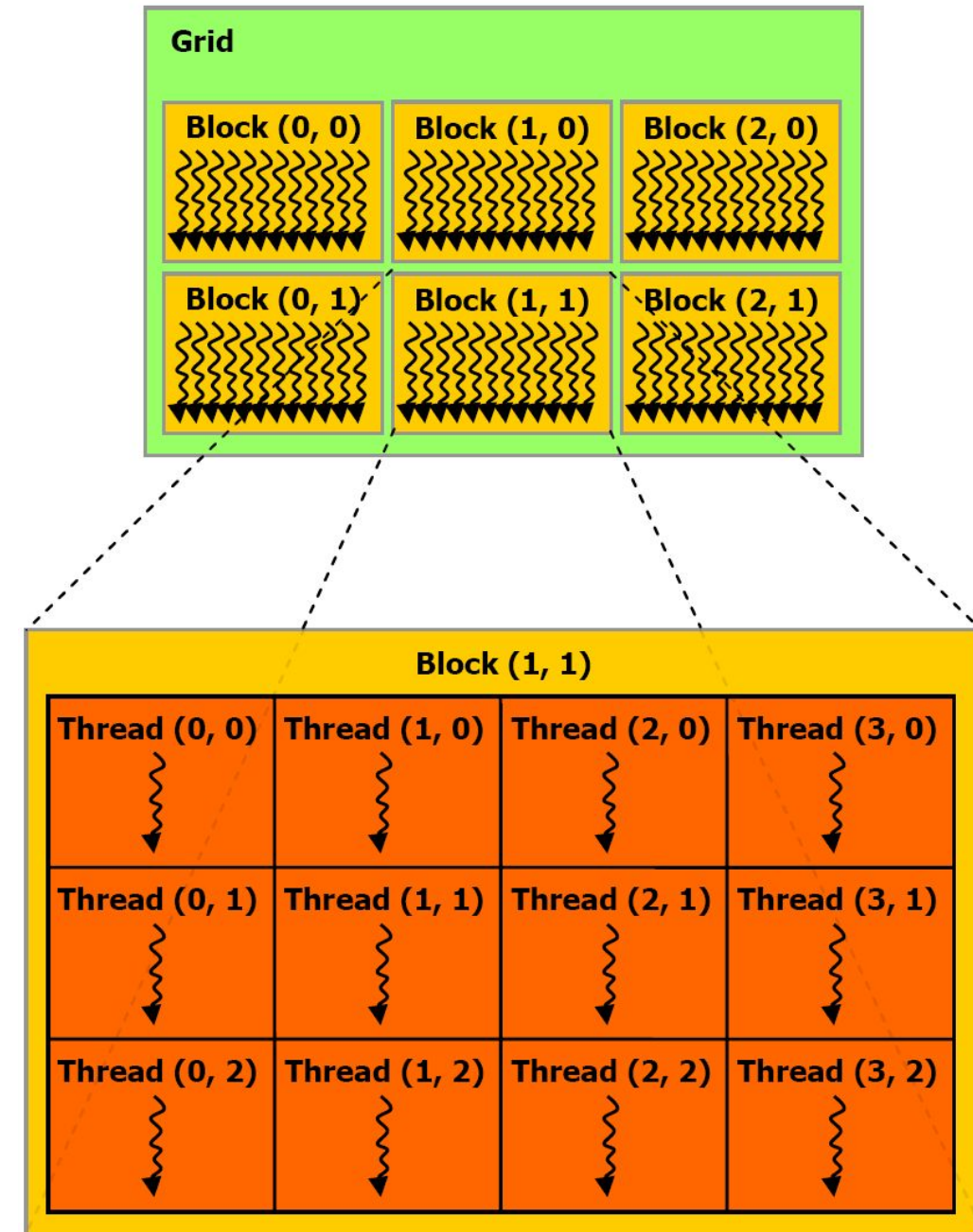
Interconnect

CPU and GPU are linked via high-speed PCIe or NVLINK interconnects.

Grid, Block, Thread



- One grid generation for one function
- Block structure could be 1D, 2D or 3D
- Thread structure could be 1D, 2D or 3D



Execution configuration

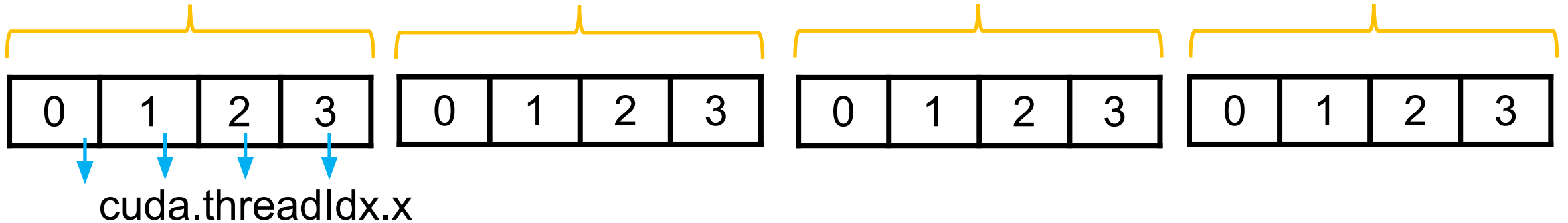
`kernel[4,4](in_array,out_array); cuda.blockDim.x=4, cuda.gridDim.x=4`

`cuda.blockIdx.x=0`

`cuda.blockIdx.x=1`

`cuda.blockIdx.x=2`

`cuda.blockIdx.x=3`

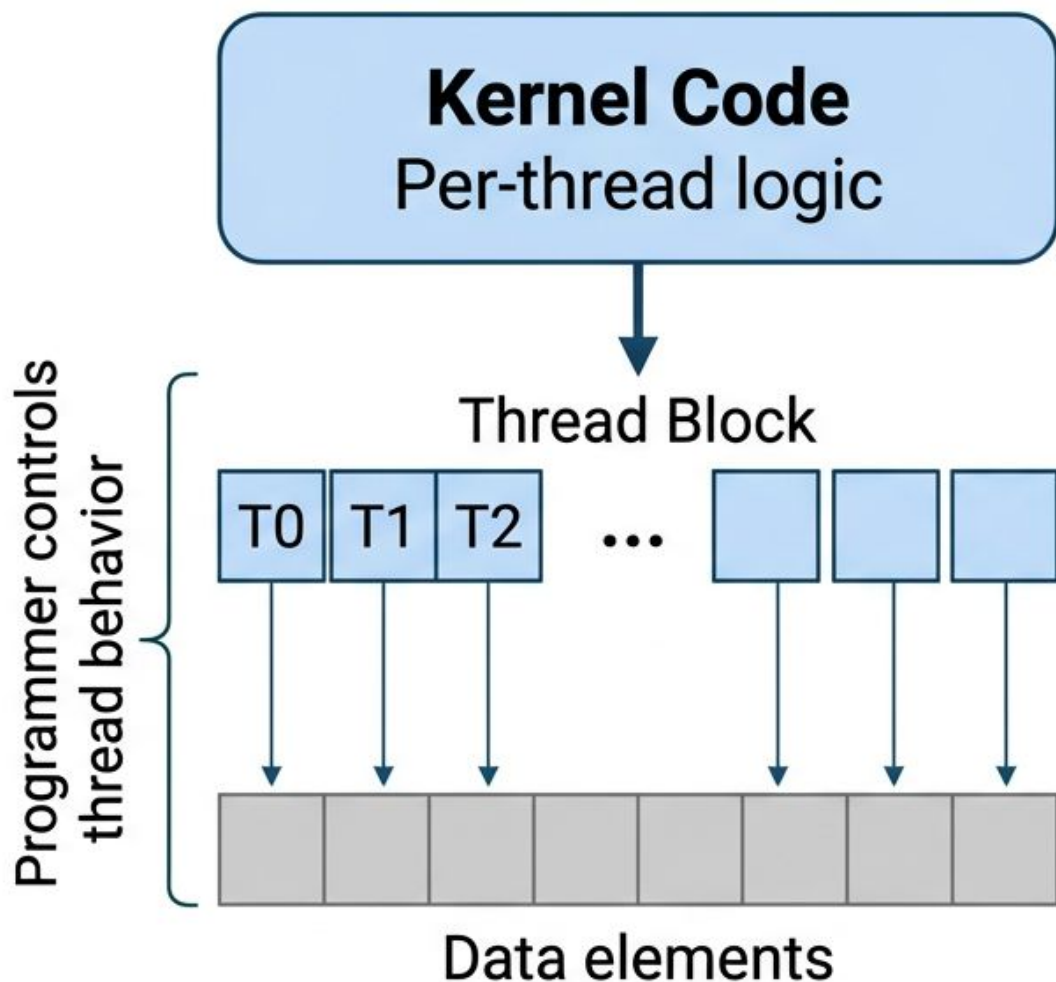


`idx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x;`

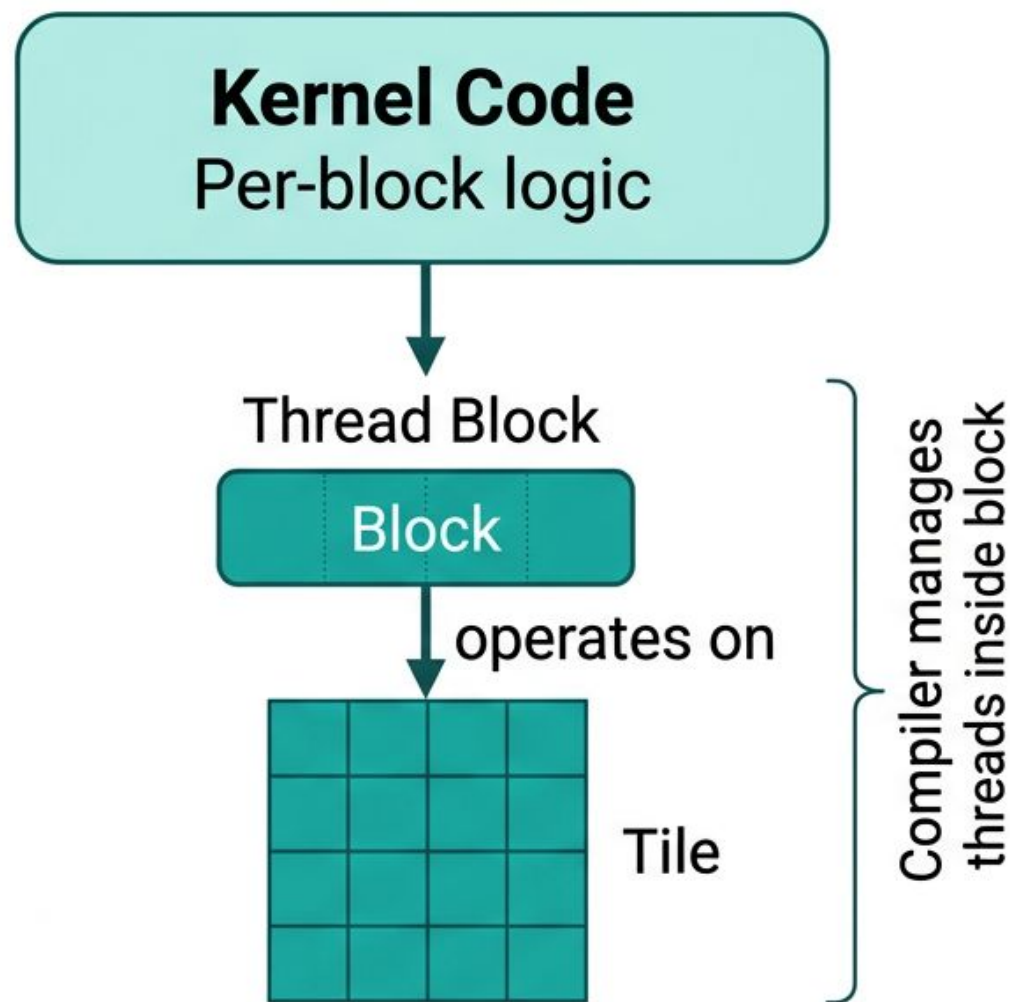
`idx=cuda.grid(1)`



SIMT Model

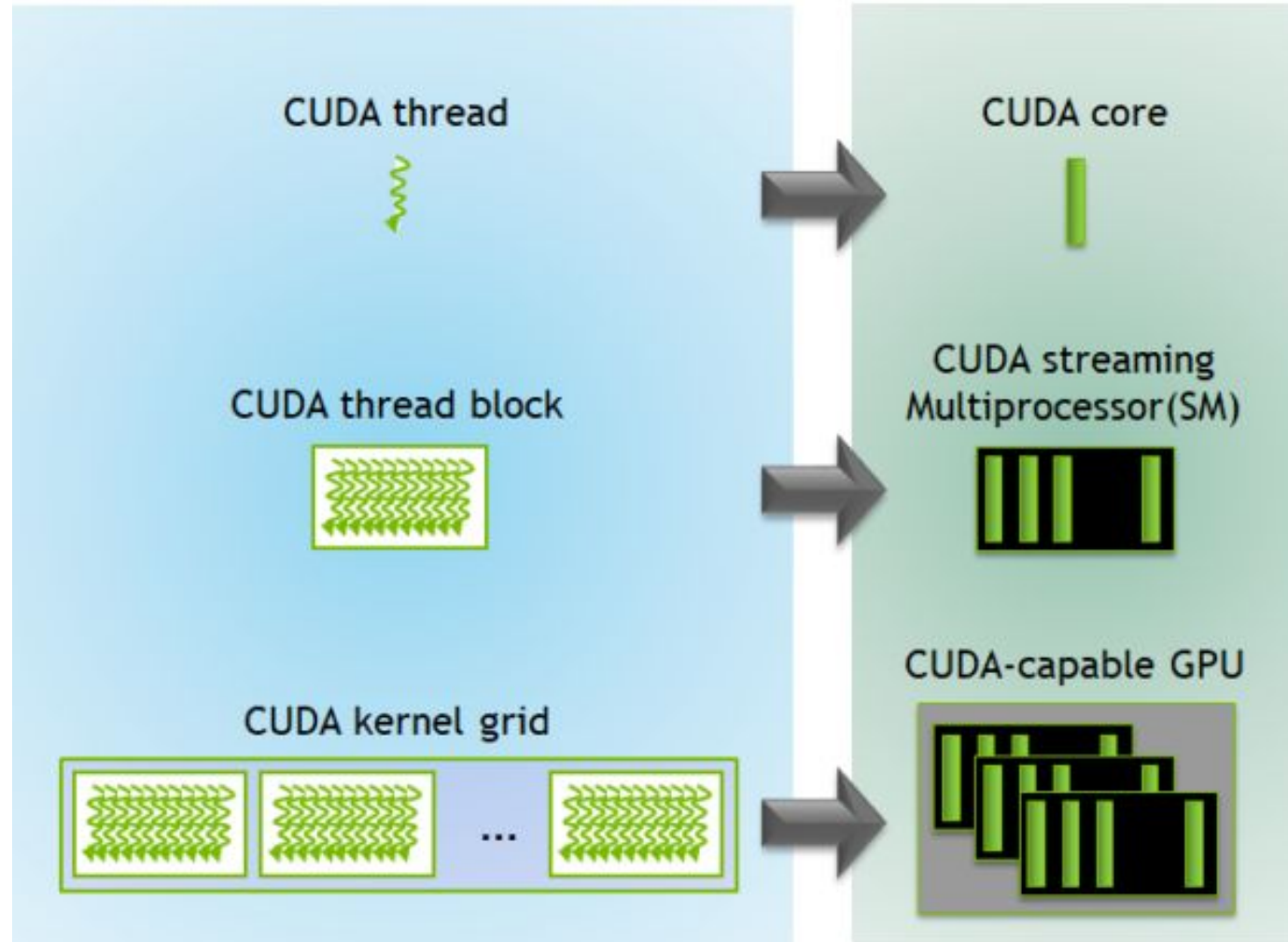


Tile Model



SIMT: programmer thinks in threads. Tile: programmer thinks in blocks and tiles.

Threads vs. Cores



AI (Arithmetic Intensity)

Definition: the number of operations performed per byte of data

Vector Addition

AI for $a[N] + b[N] = c[N]$ using F32 precision:

$$N / (3 \cdot 4)N = 1/12$$

Iconstant AI value

Matrix Multiplication

$A[N][N] * B[N][N] = C[N][N]$ with F32 precision:

$$2N^3 / (3 \cdot 4)N^2 = N/6$$

Scales proportionally with N

Complex-to-Complex FFT

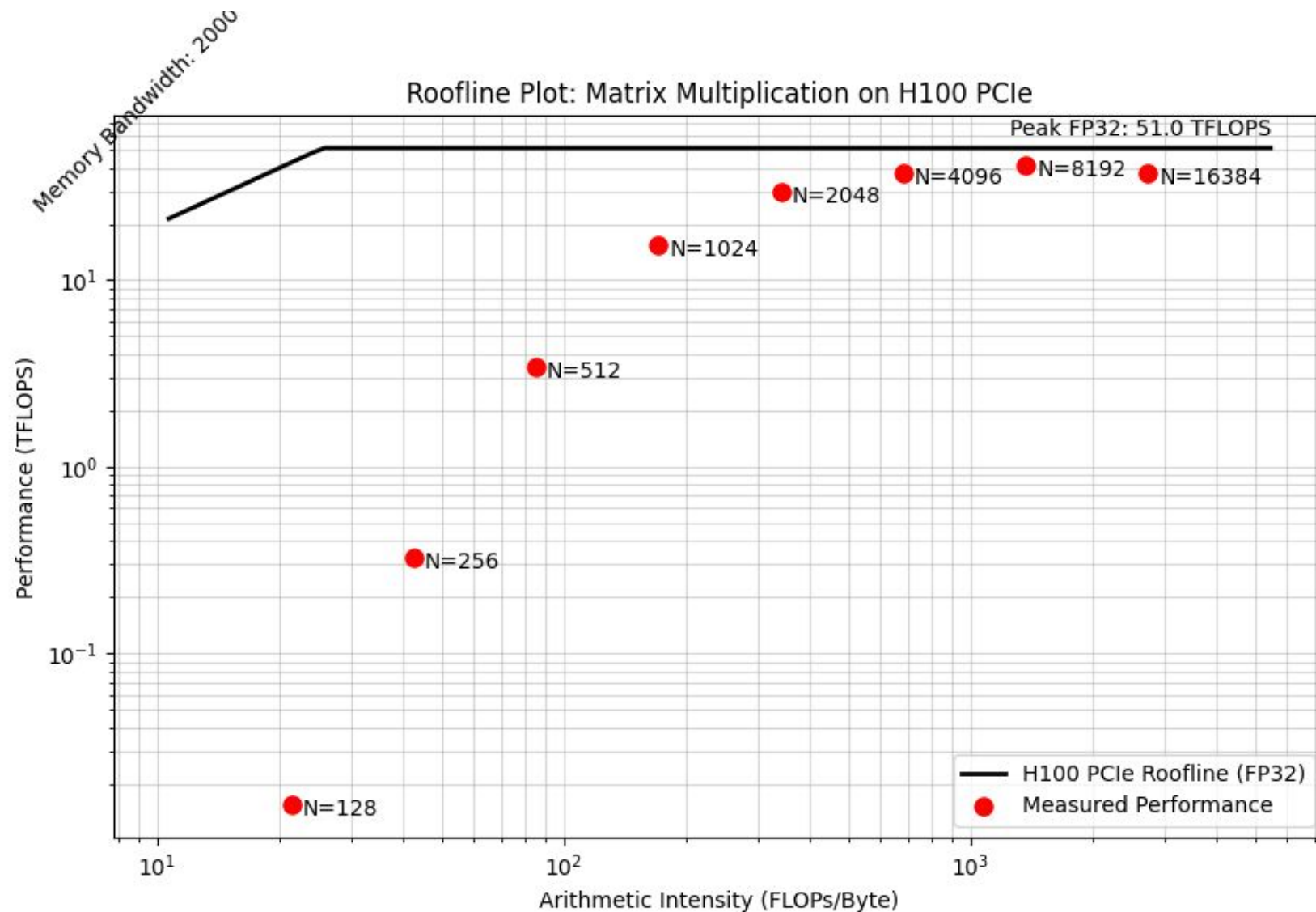
N complex inputs to N complex outputs (F32). 16N bytes for 4 complex numbers.

$$5N \log_2 N / (2 \cdot 2 \cdot 4)N = 5/16 \log_2 N$$

Scales proportionally with $\log_2 N$

Higher AI values are better suited for GPU computations

Roofline analysis for matrix-matrix multiplication



Vibe Coding

Literal Meaning: A style of development where the programmer provides high-level intent and "vibes" to AI, which then generates the underlying code.

Initiator: Popularized by Andrej Karpathy, describing a shift from syntax-heavy programming to high-level system orchestration.

Final Goal: To enable "programming at the speed of thought," where the barrier between human creativity and software execution is minimized.

Vibe Coding Tools:

- Cursor (AI-native IDE)
- GitHub Copilot
- Replit Agent
- Google Antigravity

Google Antigravity <https://antigravity.google/>

Google Antigravity

A premier agentic development platform designed to bridge the gap between complex software architecture and intuitive natural language instruction.

Antigravity CLI

A lightweight terminal-first surface for rapid environment initialization and management.

Antigravity SDK

Enables developers to prototype custom agents using simple Python scripts with minimal code.

Antigravity IDE (install this one)

A fully-featured IDE with deep codebase understanding and an agent manager.

Markdown Format

A lightweight markup language used by [GitHub](#), [Gemini](#), and many others to structure generated content and documentation.

Key Features

- Easy readability in raw text
- Converts seamlessly to HTML
- Standard for READMEs & Wikis
- Supports tables, lists, & code

Common Syntax

Header - Main Titles
Bold - Emphasis
Italic - Emphasis
- List - Bullet points
1. List - Numbered items
[Text] (url) - Hyperlinks
> Quote - Blockquotes
`code` - Inline or blocks

Lexicon: 10-node GPU Cluster

(Specifications per Node)

Compute Power

CPU: 2x Intel Xeon Gold 5118

(2.3GHz, 12 Cores each)

Graphics Processing

GPU: 4x NVIDIA Tesla V100

32GB HBM2 Memory per GPU

Networking & Memory

RAM: 192GB

100G Infiniband | 10G Ethernet

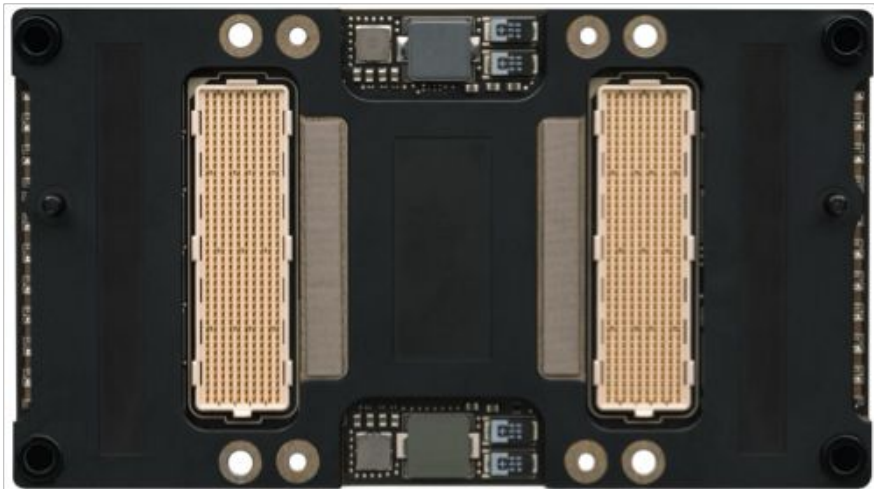
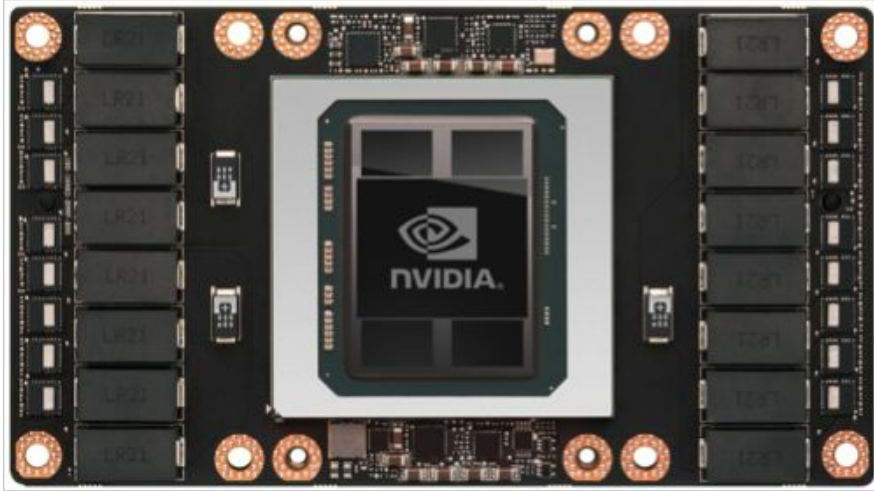
Software Stack

OS: Rocky Linux 9.6

Scheduler: Slurm Workload Manager

Tutorial Cluster for CUDA Python Lecture

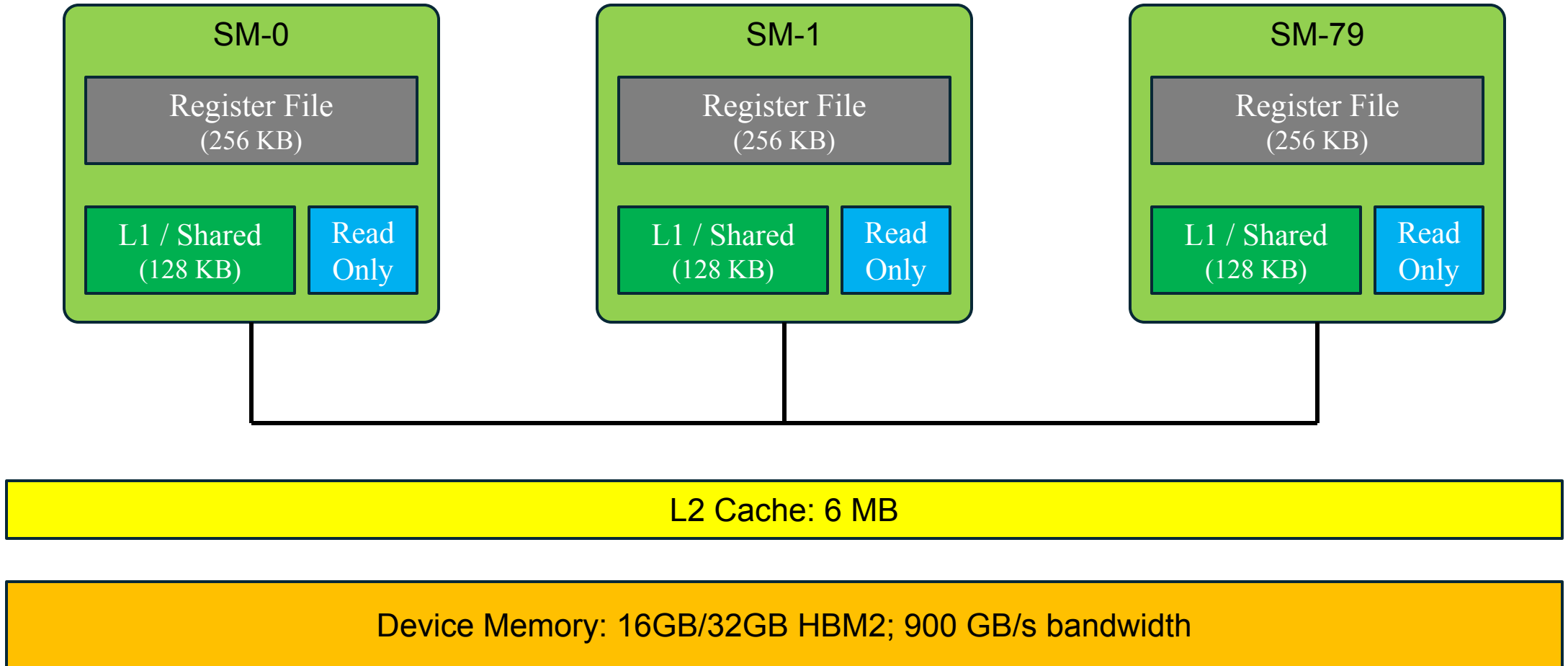
NVIDIA TESLA V100 SXM2-32GB



- Volta Architecture
 - Cores: 5140 FP32+640 Tensor
 - Memory: 32 GB HBM2, 900GB/s bandwidth
 - 2nd Gen NVLink: 300 GB/s
 - PCIe gen3: 32 GB/s bi-direction
 - Max Power: 300 W
-
- 7.8 TFLOPS of FP64
 - 15.7 TFLOPS of FP32
 - 125 TFLOPS of Tensor

Memory Hierarchy of NVIDIA Tesla V100

64 FP32 cores per SM



FP32 performance: 15.7 TFLOPS

Compute Capability	Data Center	Workstation/Consumer	Jetson
12.1		NVIDIA GB10 (DGX Spark)	
12.0	NVIDIA RTX PRO 6000 Blackwell Server Edition NVIDIA RTX PRO 4500 Blackwell Server Edition	NVIDIA RTX PRO 6000 Blackwell Workstation Edition NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition NVIDIA RTX PRO 5000 Blackwell NVIDIA RTX PRO 4500 Blackwell NVIDIA RTX PRO 4000 Blackwell NVIDIA RTX PRO 4000 Blackwell SFF Edition NVIDIA RTX PRO 2000 Blackwell GeForce RTX 5090 GeForce RTX 5080 GeForce RTX 5070 Ti GeForce RTX 5070 GeForce RTX 5060 Ti GeForce RTX 5060 GeForce RTX 5050	
11.0			Jetson T5000 Jetson T4000
10.3	NVIDIA GB300 NVIDIA B300	NVIDIA GB300 (DGX Station)	
10.0	NVIDIA GB200 NVIDIA B200		
9.0	NVIDIA GH200 NVIDIA H200 NVIDIA H100		

SSH configuration in Mac and Linux

1. Passwordless SSH Login

```
mkdir ~/.ssh; cd ~/.ssh  
ssh-keygen -t ed25519  
  
ssh-copy-id cac001@lex.kias.re.kr (replace  
cac001 with your user id)
```

3. test ssh login without password

```
ssh cac001@lex.kias.re.kr  
ssh cac001@lex01.kias.re.kr
```

2. SSH Configuration (~/.ssh/config)

```
Host lex lexicon  
Hostname lex.kias.re.kr  
User cac001 (your user id)  
  
# access Compute Nodes through lex  
Host lex0? lex10  
Hostname %h  
User cac001 (your user id)  
ProxyJump lex
```


Clone codes and Installation of Anti-Gravity IDE

1. Clone Lecture Repository

```
git clone https://github.com/jskim-kasi/lecture-kias-2026.git
```

2. Installation of Anti-Gravity IDE

```
visit https://antigravity.google/product  
install IDE (Not antigravity, not CLI)
```

nvidia-smi & module avail

- `nvidia-smi`

- `module avail`

```
EasyBuild/5.0.0 libfabric/1.18.0 python/3.12.12
```

```
autotools nodejs python/3.13.9
```

```
cmake/4.0.0 os python/3.14.0 (D)
```

```
gnu12/12.2.0 pmix/4.2.9 spack/0.23.1
```

```
gnu13/13.2.0 prun/2.2 ucx/1.18.0
```

```
gnu14/14.2.0 python/3.9.24 valgrind/3.24.0
```

```
gnu15/15.1.0 python/3.10.19
```

```
hwloc/2.12.0 python/3.11.14
```

```
# add the following line at the end of your ~/.bashrc
```

```
module load python/3.12.12
```

Python

+ Pros

- **High-productivity** dynamic programming language widely used in science, engineering, and data analytics.
- Clean and expressive syntax with standard data structures.
- Comprehensive “*batteries included*” standard library and excellent documentation.
- Broad ecosystem of libraries, professional support, and a large open community.

- Cons

- **Poor performance** for data- and computation-intensive programs.
- Efficiency-focused programmers often **rewrite innermost loops in C** and call those compiled functions from Python.

NUMPY

Package for **scientific computing** in Python.

Core Capabilities

- Multidimensional array object
- Fast operations: math, logic, sorting
- Discrete Fourier transforms
- Linear algebra & statistics
- Random simulation

Implementation

Composed of optimized, pre-compiled, **fast C codes**.

Operating Principle:

Based on *broadcasting* and *element-by-element* fashion.

```
import numpy as np
```

```
def multiply_matrices():
```

```
    # Define the matrix size
```

```
    N = 8192
```

```
    # Initialize matrices A and B with random values in float32
```

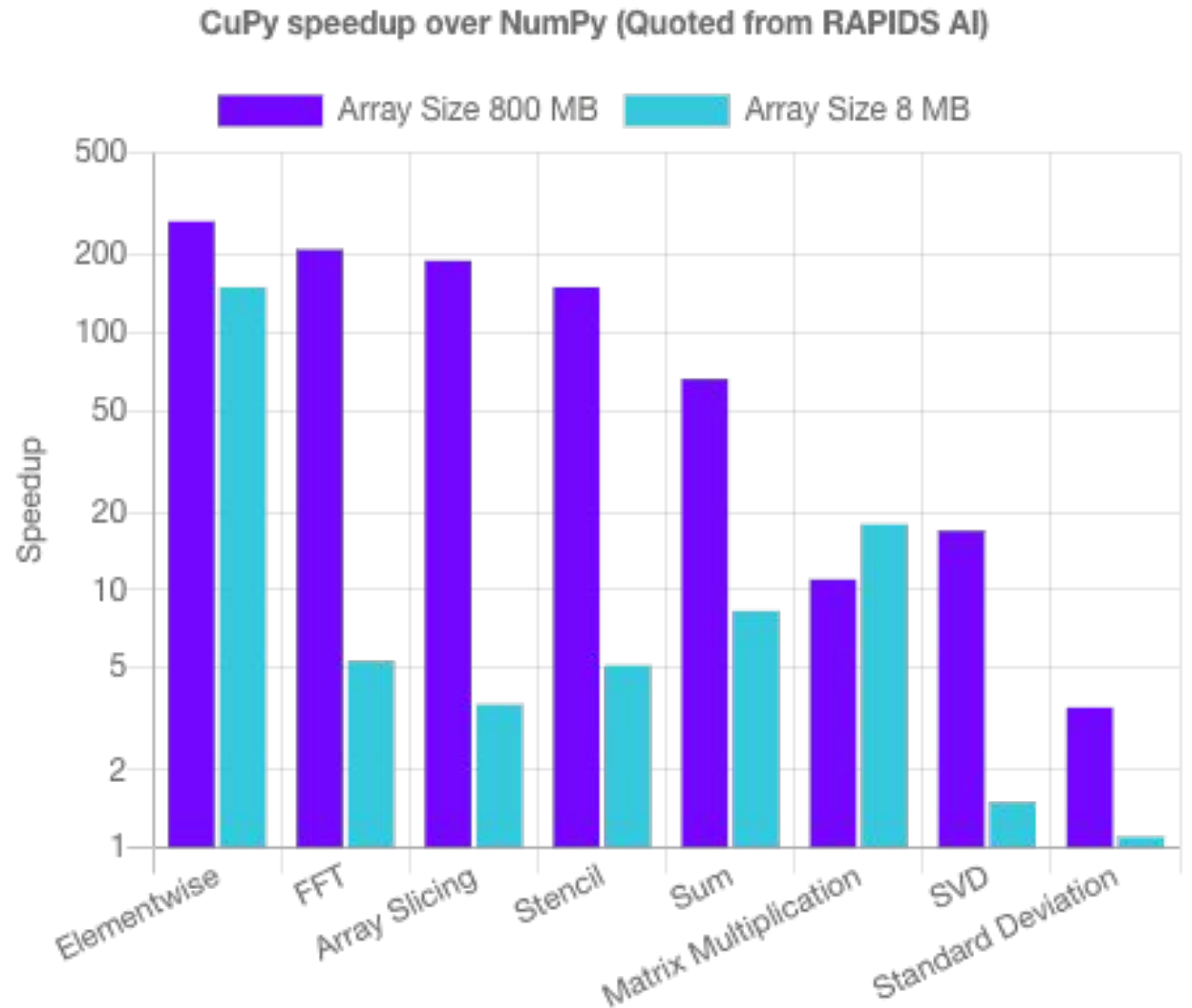
```
    A = np.random.rand(N, N).astype(np.float32)
```

```
    B = np.random.rand(N, N).astype(np.float32)
```

```
    C = A @ B
```

CuPy

- NumPy/SciPy-compatible array library for GPU-accelerated computing with Python
- CuPy utilizes NVIDIA libraries, including CUDA, cuBLAS, cuRAND, cuSOLVER, cuSPARSE, cuFFT, cuDNN, and NCCL, to make full use of GPU architecture.
- Accelerates performance by 10x-100x compared to NumPy for large-scale computations.
- <https://cupy.dev/>



NVIDIA Tesla V100 32 GB; Python 3.7.3; NumPy 1.16.4; CuPy 6.1.0

```
import cupy as cp
```

```
def multiply_matrices():
```

```
    # Define the matrix size
```

```
    N = 8192
```

```
    # Initialize matrices A and B with random values in float32
```

```
    A = cp.random.rand(N, N).astype(cp.float32)
```

```
    B = cp.random.rand(N, N).astype(cp.float32)
```

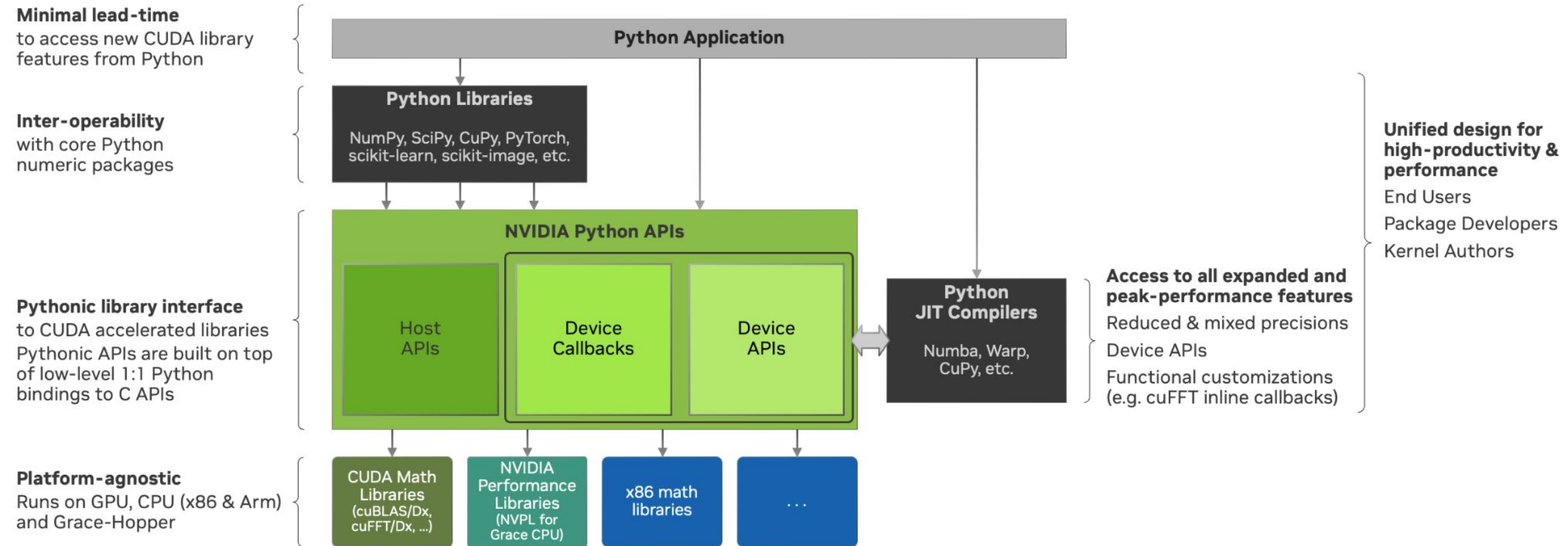
```
    C = cp.matmul(A, B)
```

nvmath-python: High-Performance Math

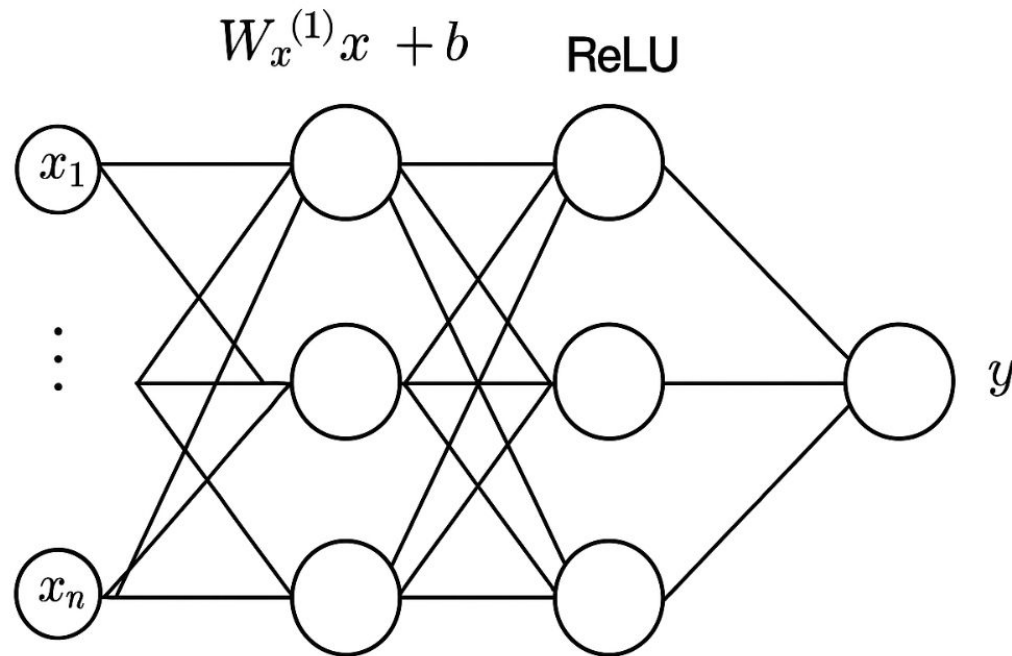
Bringing NVIDIA's high-performance math libraries to the Python ecosystem with intuitive, Pythonic APIs.

- **High-Level Pythonic APIs:** Advanced Linear Algebra (GEMM) with epilog fusion (ReLU, Bias) and N-dimensional FFTs.
- **Direct CUDA Access:** Low-level bindings for cuBLAS, cuFFT, cuRAND, and cuSOLVER.
- **Stateful Execution:** Supports object-oriented "plan-execute" patterns for maximum performance.
- **Seamless Interoperability:** Native integration with NumPy, PyTorch, and CuPy without data copies.
- **Device APIs:** Embed math directly into custom kernels using Numba or other Python compilers.

nvmath-python Architecture



Forward propagation of neural network



- $W(m,n)$: weight matrix with m outputs and n inputs
- $x(n,b)$: batch (b) of input vectors (n)
- $B(m)$ bias vector of m
- $\text{ReLU} (W(m,n) X (n,b) + B(m))$
 - $\text{ReLU}(x) = \max(0,x)$

implementation of each of three operations

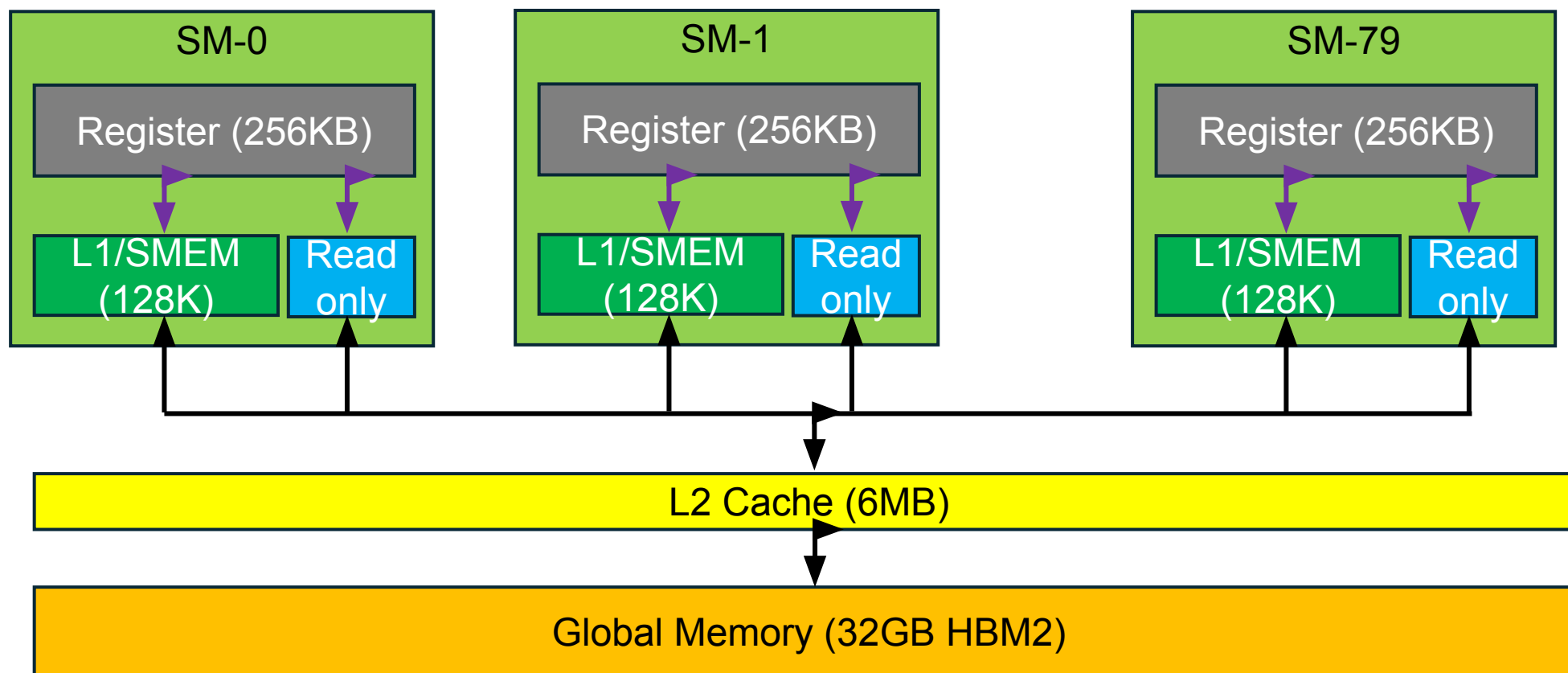
```
from nvmath.linalg.advanced import Matmul
```

```
mm = Matmul(weights, x)  
mm.plan()
```

```
def forward():  
    y = mm.execute()  
    y += bias[:,cupy.newaxis]  
    y[y < 0] = 0  
    return y
```

Epilogs

- *Epilogs* are operations that can be fused with the mathematical operation being performed, like FFT or matrix multiplication



implementation with epilog

```
from nvmath.linalg.advanced import Matmul  
from nvmath.linalg.advanced import MatmulEpilog
```

```
mm = Matmul(weights, x)  
mm.plan(epilog=MatmulEpilog.RELU_BIAS,epilog_inputs={"bias":  
bias})
```

```
def forward():  
    y = mm.execute()  
    return y
```

Writing kernels in Python using Numba

- **GPU as Device:** Numba compiles a subset of Python code directly into CUDA kernels executed on the GPU.
- **Execution Model:** Massive parallelism through a hierarchy of Grids, Blocks, and Threads.
- **Memory Access:** Explicit management of Global, Shared, and Local memory for performance.
- **Asynchronous Execution:** Kernels are launched from the Host (CPU) and return control immediately.
- Source: <https://nvidia.github.io/numba-cuda/index.html>

CUDA kernel for vector addition

```
@cuda.jit
def vector_add_kernel(x, y, out):
    # Get the global position of the current thread
    idx = cuda.grid(1)

    # Check if the thread index is within array bounds
    if idx < out.size:
        out[idx] = x[idx] + y[idx]
```

CUDA kernel for matrix multiplication

```
@cuda.jit
def matmul_kernel(A, B, C):
    row, col = cuda.grid(2)

    if row < C.shape[0] and col < C.shape[1]:
        tmp = 0.0
        for k in range(A.shape[1]):
            tmp += A[row, k] * B[k, col]
        C[row, col] = tmp
```


cuTile: Programming with Cooperative Tiles

lowest level of parallelism to the compiler and built-in operations

- **Hierarchical Abstraction:** cuTile introduces a virtual instruction set that maps logical tiles to physical GPU hardware units like SMs and Tensor Cores.
- **Tiled Execution Model:** Algorithms are expressed as operations on localized data fragments rather than global memory sweeps.
- **Data Movement:** Explicitly manages the "load-compute-store" pipeline between Global Memory, Shared Memory, and Registers.
- **Hardware Independence:** Write high-level Python code that automatically targets Blackwell and Hopper architectures.
- **Supported GPU Architectures:** A GPU with compute capability 10.x or 12.x (**Blackwell and Hopper architectures**).

Key Capabilities

- Optimized for Matrix Multiply-Accumulate (MMA)
- Support for overlapping memory copies with compute
- Simplified synchronization between threads in a tile

SIMT Matrix Multiplication (Numba 2D)

2D Thread Grid Execution: Each individual thread computes exactly one element of the output matrix C.

Core Model Concepts

- **2D Thread Grid:**

The execution configuration divides the 2D output matrix C into a grid of 2D thread blocks (typically 16x16 threads per block).

- **1:1 Mapping:**

Each thread calculates its row and col using `cuda.grid(2)`. There is a direct mapping: Thread (row, col) calculates element `C[row, col]`.

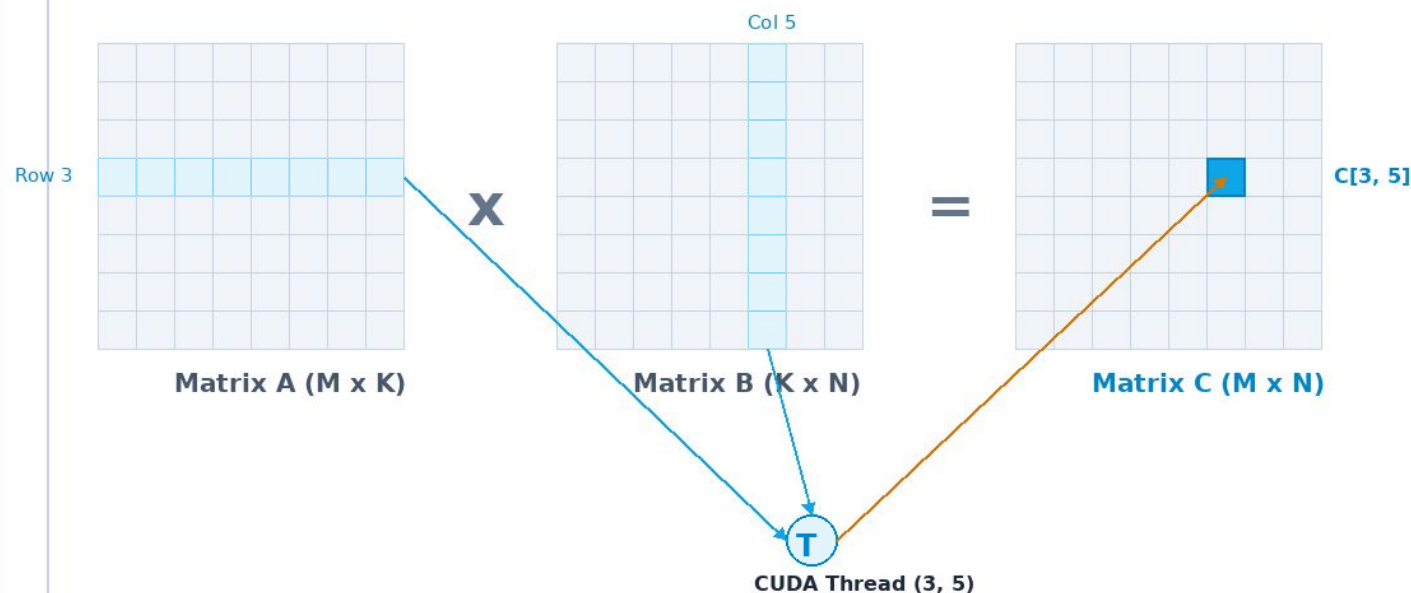
- **Dot Product Loop:**

The thread executes a sequential loop over the shared dimension K, loading elements along row in A and col in B to calculate the sum.

- **Memory Bounds:**

Threads check `row < M` and `col < N` to ensure they don't read or write out-of-bounds.

Naive Matrix Multiplication Execution Diagram



Dot Product Loop $C[3, 5] = A[3, 0] * B[0, 5] + A[3, 1] * B[1, 5] + A[3, 2] * B[2, 5] + \dots + A[3, 7] * B[7, 5]$

TILE Matrix Multiplication (cuTile)

2D Block Tiling with Tensor Cores: Cooperative blocks load sub-matrix tiles and execute matrix multiply-accumulate.

Tiled Model Concepts

- **Block-Level Tiles:**

Thread Blocks (CTAs) operate collectively on 2D memory slices (tiles) of shape $\text{TILE_M} \times \text{TILE_N}$ (typically 16x16 or 32x32).

- **Collective Loading:**

The entire block coordinates to load x_{tile} and y_{tile} collectively from global memory using `ct.load`, optimizing global memory coalescing.

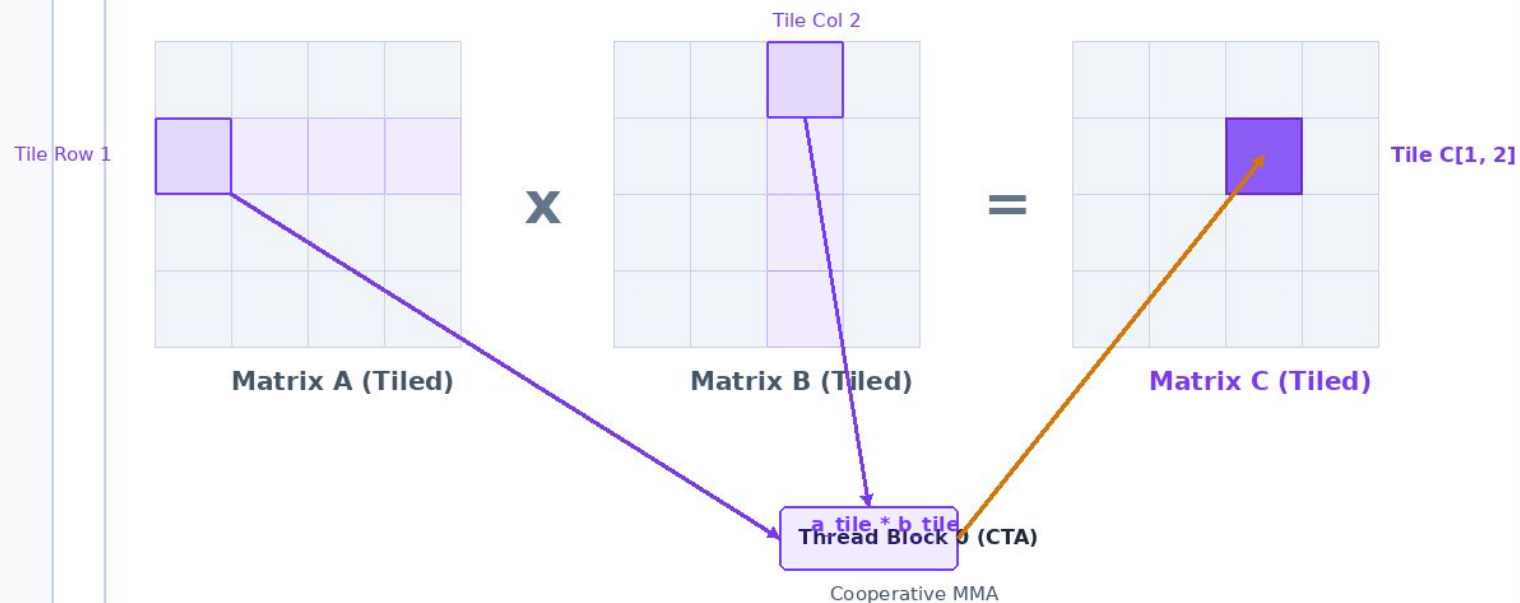
- **Tensor Core MMA:**

Rather than element-wise scalar operations, the block executes `ct.mma(a_tile, b_tile, accumulator)` utilizing Tensor Core instructions.

- **Implicit Coops:**

Threads work cooperatively under the hood; indexing complexity, boundary handling, and synchronization are abstracted by cuTile compiler.

Tiled Block Execution Diagram



Tiled Loop Step k `accumulator += ct.mma(A_tile[1, k], B_tile[k, 2], accumulator)`

Summary and Recommendations

- Calculate AI (Arithmetic intensity) of your application.
 - Higher AI values are better suited for GPU computations
- Make a Python script using NumPy and check performance using CPU
- Convert the Python script using CuPy and check performance using GPU
- If your application is good enough to use linear algebra or FFT with epilogue, then modify the Python script using nvmath.
- If you have a Blackwell/Hopper GPU, then try cuTile. cuTile is easy to write a custom kernel and will be getting popular.
- If you really want fully optimized code, try to use CUDA C++
 - CUDA C++ provides maximum performance and control, full access to GPU memory hierarchy and hardware features, and is best for production, real-time, and finely tuned HPC applications